

PYTHON DATA ANALYTICS

NumPy and Pandas

Domain: Data Analytics

Tools Used: Python, NumPy, Pandas, Google Colab

PROJECT TITLE

Data Analysis Using NumPy and Pandas

PROJECT OVERVIEW

This project focuses on performing basic data analysis and manipulation using Python libraries **NumPy** and **Pandas**.

The project consists of three major areas:

1. Numerical analysis of weekly temperature data using NumPy.
2. Data manipulation and indexing using Pandas Series.
3. Data exploration, filtering, aggregation, and manipulation using a Pandas Data Frame.

The assignment demonstrates fundamental data analytics techniques such as array operations, indexing, slicing, filtering, aggregation, grouping, and data modification.

OBJECTIVES

The main objectives of this project are:

- To understand and work with NumPy arrays for numerical computations.
- To create and manipulate one-dimensional and two-dimensional arrays.
- To perform indexing and slicing operations on NumPy arrays.
- To calculate maximum, minimum, mean, and temperature conversions.
- To create and manipulate Pandas Series.
- To understand the difference between `loc` and `iloc` indexing.
- To apply Boolean filtering on Pandas Series and Data Frames.
- To create and explore Pandas Data Frames.
- To perform grouping and aggregation operations.
- To modify, add, and remove data from a Data Frame.
- To develop practical data-handling skills using Python.

TECHNOLOGIES AND LIBRARIES USED

Programming Language

Python

Python is used as the primary programming language for performing data analysis and manipulation.

Libraries

NumPy

NumPy is used for:

- Numerical computations
- Array creation
- Array indexing and slicing
- Statistical calculations
- Mathematical operations

Pandas

Pandas is used for:

- Series creation
 - Data Frame creation
 - Data filtering
 - Data selection
 - Data aggregation
 - Grouping
 - Data modification
-

DATASET DESCRIPTION

The project uses two types of datasets.

Temperature Dataset

The temperature dataset contains daily average temperatures recorded over two weeks.

Week 1

Day Temperature (°C)

Day 1 22.5

Day Temperature (°C)

Day 2	25.3
Day 3	20.8
Day 4	23.4
Day 5	26.1
Day 6	24.8
Day 7	21.9

Week 2

Day Temperature (°C)

Day 1	19.2
Day 2	22.5
Day 3	21.3
Day 4	24.0
Day 5	23.5
Day 6	22.8
Day 7	20.1

Transaction Dataset

The transaction dataset contains information about transactions, product categories, regions, and transaction amounts.

Transaction ID Product Category Region Amount

101	Electronics	North	200
102	Clothing	South	150
103	Electronics	North	300
104	Furniture	East	450
105	Clothing	West	200
106	Electronics	North	250
107	Furniture	East	300
108	Clothing	West	180
109	Furniture	South	350
110	Electronics	North	400

IMPLEMENTATION

Importing Libraries

The following libraries were imported:

```
import numpy as np
import pandas as pd
```

`numpy` is used for numerical array operations, while `pandas` is used for structured data analysis.

NUMPY ARRAY OPERATIONS

Creating a One-Dimensional Array

A NumPy array named `temperatures_w1` was created to store the Week 1 temperature values.

```
temperatures_w1 = np.array([
    22.5, 25.3, 20.8, 23.4,
    26.1, 24.8, 21.9
])
```

The array contains seven temperature values representing seven days of Week 1.

Inspecting Array Properties

The following properties were examined:

- Shape
- Data type
- Number of elements

```
print(temperatures_w1.shape)
print(temperatures_w1.dtype)
print(temperatures_w1.size)
```

Expected Result

- Shape: `(7,)`
- Data Type: `float64`
- Number of Elements: 7

The shape `(7,)` indicates that the array is one-dimensional and contains seven values.

TEMPERATURE CONVERSION

The Celsius temperatures were converted into Fahrenheit using the formula:

$$\text{Fahrenheit} = (\text{Celsius} \times 9/5) + 32$$

Python implementation:

```
temperatures_fahrenheit = (temperatures_w1 * 9/5) + 32
```

This demonstrates NumPy's ability to perform mathematical operations on an entire array without using individual loops.

TEMPERATURE STATISTICAL ANALYSIS

The maximum, minimum, and mean temperatures were calculated using NumPy functions.

```
np.max(temperatures_w1)
np.min(temperatures_w1)
np.mean(temperatures_w1)
```

Results

- **Maximum temperature:** 26.1°C
- **Minimum temperature:** 20.8°C
- **Average temperature:** approximately 23.54°C

The results provide a basic understanding of the temperature variation during Week 1.

ARRAY SLICING AND INDEXING

NumPy slicing was used to extract specific portions of the temperature data.

First Three Days

```
first_three_days = temperatures_w1[:3]
```

Result:

```
[22.5 25.3 20.8]
```

Weekend

```
weekend = temperatures_w1[-2:]
```

Result:

```
[24.8 21.9]
```

Middle Three Days

```
middle_three_days = temperatures_w1[2:5]
```

Result:

```
[20.8 23.4 26.1]
```

This demonstrates how slicing can be used to extract specific sections of an array.

CREATING A TWO-DIMENSIONAL ARRAY

A two-dimensional NumPy array named `temperatures` was created to store temperature data for both weeks.

```
temperatures = np.array([
    [22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9],
    [19.2, 22.5, 21.3, 24.0, 23.5, 22.8, 20.1]
])
```

Each row represents one week, while each column represents a day.

TWO-DIMENSIONAL ARRAY PROPERTIES

The following properties were inspected:

```
temperatures.shape
temperatures.dtype
temperatures.size
```

Results

- **Shape:** (2, 7)
- **Data Type:** float64
- **Total Elements:** 14

The shape (2, 7) indicates that the array contains 2 rows and 7 columns.

EXTRACTING WEEKLY AND WEEKEND TEMPERATURES

The temperature values for each week were extracted using indexing.

```
week1 = temperatures[0]
week2 = temperatures[1]
```

Weekend temperatures were extracted using:

```
week1_weekend = temperatures[0, -2:]
week2_weekend = temperatures[1, -2:]
```

This demonstrates two-dimensional indexing and slicing using NumPy.

PANDAS SERIES

Creating a Pandas Series

A Pandas Series named `marks` was created using marks and custom index labels.

```
marks = pd.Series(
    [95, 92, 89, 85, 80],
    index=['Rank1', 'Rank2', 'Rank3', 'Rank4', 'Rank5']
)
```

The custom labels represent the rank of each student.

SERIES INDEXING AND SLICING

Accessing the First Rank

The `iloc` accessor was used to access the first value based on integer position.

```
marks.iloc[0]
```

Result:

```
95
```

Retrieving Top Three Ranks

The `loc` accessor was used with index labels.

```
marks.loc[['Rank1', 'Rank2', 'Rank3']]
```

Retrieving Third Rank

```
marks.iloc[2]
```

Result:

```
89
```

Filtering Marks Greater Than 90

A Boolean mask was applied:

```
marks[marks > 90]
```

This returns the students whose marks are greater than 90.

MANIPULATING THE PANDAS SERIES

Updating Rank 1

The first student's mark was changed from 95 to 100.

```
marks.loc['Rank1'] = 100
```

Removing Rank 5

The last rank was removed using:

```
marks = marks.drop('Rank5')
```

Calculating CGPA

CGPA was calculated by dividing each mark by 10.

```
cgpa = marks / 10
```

This demonstrates arithmetic operations directly on a Pandas Series.

PANDAS DATAFRAME

Creating the Transaction DataFrame

A Pandas DataFrame named `transactions` was created using transaction information.

```
transactions = pd.DataFrame({
    'TransactionID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'ProductCategory': [
        'Electronics', 'Clothing', 'Electronics', 'Furniture',
        'Clothing', 'Electronics', 'Furniture', 'Clothing',
        'Furniture', 'Electronics'
    ],
    'Region': [
        'North', 'South', 'North', 'East', 'West',
        'North', 'East', 'West', 'South', 'North'
    ],
    'Amount': [
        200, 150, 300, 450, 200,
        250, 300, 180, 350, 400
    ]
})
```

DATA EXPLORATION

Several DataFrame exploration techniques were performed.

Head

The first five rows were displayed using:

```
transactions.head()
```

Tail

The last five rows were displayed using:

```
transactions.tail()
```

Shape

```
transactions.shape
```

The initial DataFrame contains:

10 rows × 4 columns

Column Names

```
transactions.columns
```

The columns are:

- TransactionID
- ProductCategory
- Region
- Amount

Data Types

```
transactions.dtypes
```

This was used to identify the data type of each column.

SELECTING SPECIFIC COLUMNS

Only the ProductCategory and Amount columns were selected:

```
transactions[['ProductCategory', 'Amount']]
```

This demonstrates column selection in Pandas.

RETRIEVING THE LAST THREE COLUMNS

The last three columns were retrieved using `iloc`:

```
transactions.iloc[:, -3:]
```

This demonstrates positional indexing in Pandas DataFrames.

DATA FILTERING

Transactions were filtered based on two conditions:

- Region should be North
- Amount should be greater than 200

```
filtered_transactions = transactions[
    (transactions['Region'] == 'North') &
    (transactions['Amount'] > 200)
]
```

The matching transactions include Transaction IDs:

- 103
- 106
- 110

This demonstrates the use of multiple Boolean conditions in Pandas.

PRODUCT CATEGORY ANALYSIS

The number of transactions in each product category was calculated using:

```
transactions['ProductCategory'].value_counts()
```

Result

Product Category Count

Electronics	4
Clothing	3
Furniture	3

Electronics has the highest number of transactions in the dataset.

UNIQUE REGIONS

Unique regions were identified using:

```
transactions['Region'].unique()
```

The dataset contains the following regions:

- North
 - South
 - East
 - West
-

REGIONAL AVERAGE ANALYSIS

The average transaction amount for each region was calculated using `groupby()` and `mean()`.

```
region_mean = transactions.groupby('Region')['Amount'].mean()
```

Results

Region Average Amount

East	375.00
North	287.50
South	250.00
West	190.00

The **East region** has the highest average transaction amount, while the **West region** has the lowest.

DATAFRAME MANIPULATION

Updating Transaction Amount

The amount for TransactionID 102 was changed from 150 to 165.

```
transactions.loc[
    transactions['TransactionID'] == 102,
    'Amount'
] = 165
```

ADDING A DISCOUNT COLUMN

A new column named `Discount` was created by calculating 10% of the transaction amount.

```
transactions['Discount'] = transactions['Amount'] * 0.10
```

This demonstrates how calculated columns can be created using Pandas.

REMOVING A TRANSACTION

TransactionID 109 was removed from the DataFrame.

```
transactions = transactions[
    transactions['TransactionID'] != 109
]
```

After this operation, the DataFrame contains **9 transactions**.

DELETING THE DISCOUNT COLUMN

The temporary `Discount` column was removed using:

```
transactions = transactions.drop(columns=['Discount'])
```

The final DataFrame contains the original four columns:

- TransactionID
- ProductCategory

- Region
 - Amount
-

KEY LEARNINGS

Through this assignment, the following concepts were practiced:

NumPy

- Creating one-dimensional arrays
- Creating two-dimensional arrays
- Array properties
- Mathematical operations
- Statistical functions
- Indexing
- Slicing
- Multi-dimensional array manipulation

Pandas Series

- Creating Series
- Custom indexing
- `loc`
- `iloc`
- Boolean filtering
- Updating values
- Removing entries
- Arithmetic operations

Pandas DataFrame

- DataFrame creation
- Data exploration
- `head()` and `tail()`
- Shape and data types
- Column selection
- Positional indexing
- Boolean filtering
- Value counts
- Unique values
- Grouping
- Mean aggregation
- Updating records
- Creating calculated columns
- Removing rows and columns

CONCLUSION

This project provided practical experience in using **NumPy and Pandas for data analysis**.

NumPy was used to analyze temperature data through array creation, mathematical calculations, statistical analysis, indexing, and slicing. Pandas was then used to work with structured student marks and transaction data.

The project demonstrated important data analytics techniques such as **data exploration, filtering, aggregation, grouping, indexing, and data manipulation**. These techniques form a strong foundation for performing more advanced data analysis and visualization using Python.

Overall, the assignment helped develop practical skills in handling and analyzing structured datasets using Python and its popular data analytics libraries.